

SI3003 - Inteligencia Artificial

CLASE 6 – APRENDIZAJE SUPERVISADO: ÁRBOLES DE DECISIÓN



Aprendizaje: panorama general

Aprender es el proceso de mejorar el desempeño de un agente a partir de la experiencia (RL, la clase pasada, fue un ejemplo).

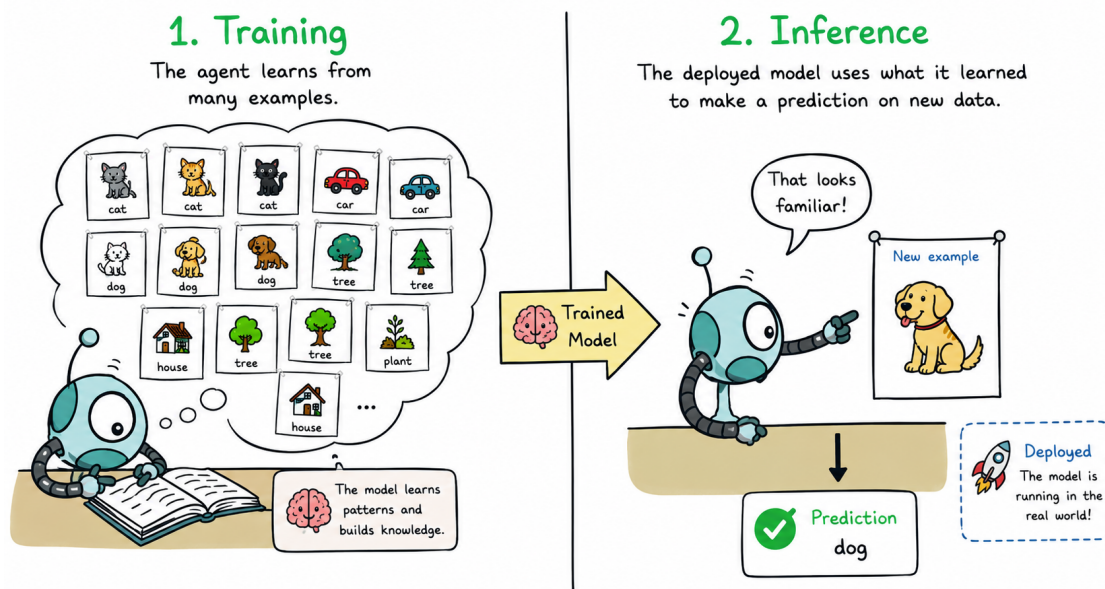
Hoy vemos:

- La idea general: *generalización a partir de la experiencia*.
- **Aprendizaje supervisado**: clasificación con árboles de decisión; una primera mirada a regresión lineal.

¿Por qué aprender?

- El bebé, asediado a la vez por ojos, oídos, nariz, piel y entrañas, siente todo como una gran confusión zumbante y floreciente...

Aprender es esencial en entornos desconocidos — cuando quien diseña al agente no puede saberlo todo de antemano.



¿Por qué aprender?

En vez de intentar producir un programa que simule la mente adulta, ¿por qué no producir uno que simule la de un niño? Si este se somete luego a un proceso de educación adecuado, se obtendría el cerebro adulto. El cerebro del niño es como un cuaderno tal como se compra en la papelería: poco mecanismo y muchas hojas en blanco.

Aprender es útil como método de construcción de sistemas — exponer el sistema a la realidad en lugar de intentar programarlo a mano. Además, ¡los humanos podemos saber hacer algo sin saber explicar cómo!



¿Cómo aprender?



Preguntas clave al construir un agente que aprende

- ¿Cuál es el diseño del agente que va a implementar el desempeño deseado?
- ¿Qué pieza del sistema quiero mejorar y cómo está representada?
- ¿Qué datos hay disponibles relevantes para esa pieza? (¿conocemos las respuestas correctas?)
- ¿Qué conocimiento previo ya está disponible?

Tipos de aprendizaje

Agent design	Component	Representation	Feedback	Knowledge
MCTS search	Evaluation function	Linear polynomial	Win / loss	Rules of the game
MDP agent	Transition model observable environment	Transition matrix	Action outcomes	Available actions, possible states
Utility-based patient monitor	Physiology / sensor model	Dynamic Bayesian network	Observation sequences	Human physiology; sensor design
Chatbot	Context-based word predictor	Transformer deep neural network	Known next word	Tokenization, vector space

Different agent designs require different representations, feedback signals, and prior knowledge.

- **Aprendizaje supervisado:** respuesta correcta para cada instancia de entrenamiento.
- **Aprendizaje por refuerzo:** secuencia de recompensas, sin respuestas correctas (Clase 5).
- **Aprendizaje no supervisado:** "encontrarle sentido a los datos", sin ninguna etiqueta.

Aprendizaje supervisado

Queremos aprender una función desconocida $f: \mathcal{X} \rightarrow \mathcal{Y}$

1. Datos de entrenamiento

Conjunto de ejemplos etiquetados:

$$D = \{(x_j, y_j)\}_{j=1}^n$$

donde $y_j = f(x_j)$



$x_1 \rightarrow y_1 = f(x_1)$
"jirafa"



$x_2 \rightarrow y_2 = f(x_2)$
"perro"



$x_3 \rightarrow y_3 = f(x_3)$
"carro"

$\vdots$

$\vdots$

$\vdots$

$\vdots$



$x_n \rightarrow y_n = f(x_n)$
"elefante"

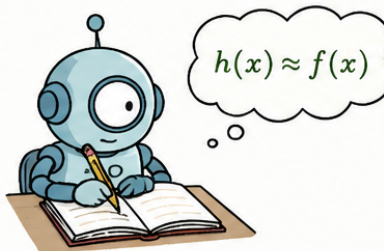
Ejemplo:

x_j : imagen

y_j : etiqueta (clase) o valor real

2. Aprendizaje

A partir de D , el algoritmo construye una hipótesis h que aproxima a f :



Hipótesis aprendida
 h

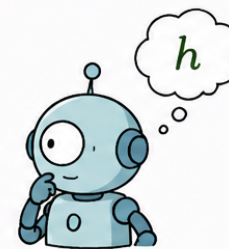
Objetivo: generalizar bien a ejemplos nunca vistos.

3. Generalización (prueba)

Para un nuevo ejemplo x_{nuevo} (no visto en entrenamiento):



x_{nuevo}



Predicción
 $\hat{y} = h(x_{\text{nuevo}})$

Esperamos que $\hat{y} \approx f(x_{\text{nuevo}})$
(la etiqueta verdadera).

Tipos de aprendizaje

Clasificación

y toma valores discretos (categorías).

Ej.: "gato", "perro", "jirafa"



...

Regresión

y toma valores continuos (reales).

Ej.: precio, temperatura, peso

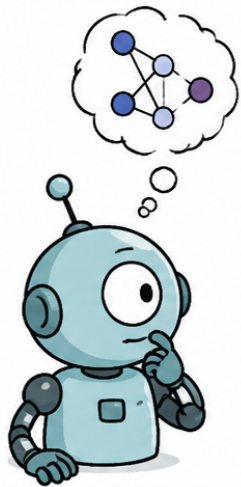


...



Existen muchas formas posibles para h : modelos lineales, regresión logística, redes neuronales, árboles de decisión, vecino-más-cercano, gramáticas, y muchas más.

Ejemplo de clasificación



x



$f(x)$

giraffe

giraffe

giraffe

llama

llama

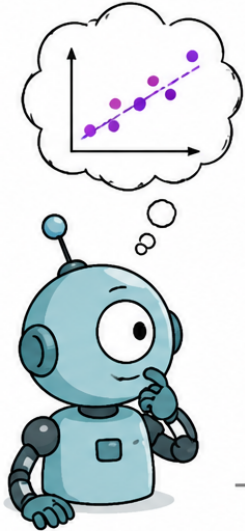
llama

$x =$



$f(x) = ?$

Ejemplo de regresión



x



área: 80 m²
hab.: 2
edad: 15 años



área: 120 m²
hab.: 3
edad: 8 años



área: 180 m²
hab.: 4
edad: 3 años



área: 200 m²
hab.: 4
edad: 2 años



área: 150 m²
hab.: 3
edad: 10 años

$f(x)$

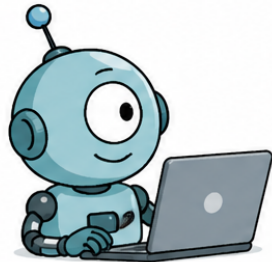
\$280 millones

\$420 millones

\$610 millones

\$750 millones

\$530 millones



$x =$



área: 135 m²
hab.: 3
edad: 5 años

$f(x) = ?$



Valor continuo
(regresión)

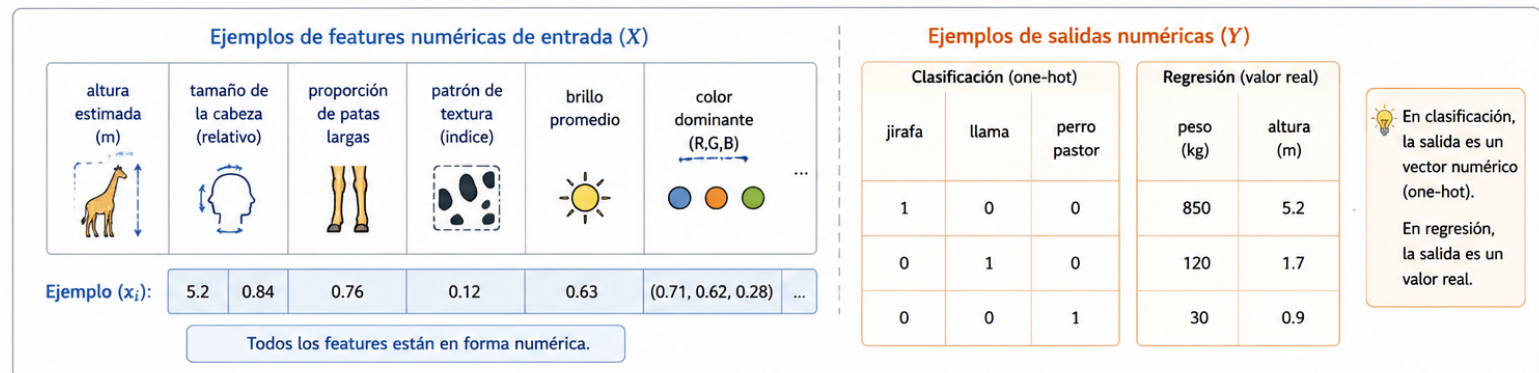
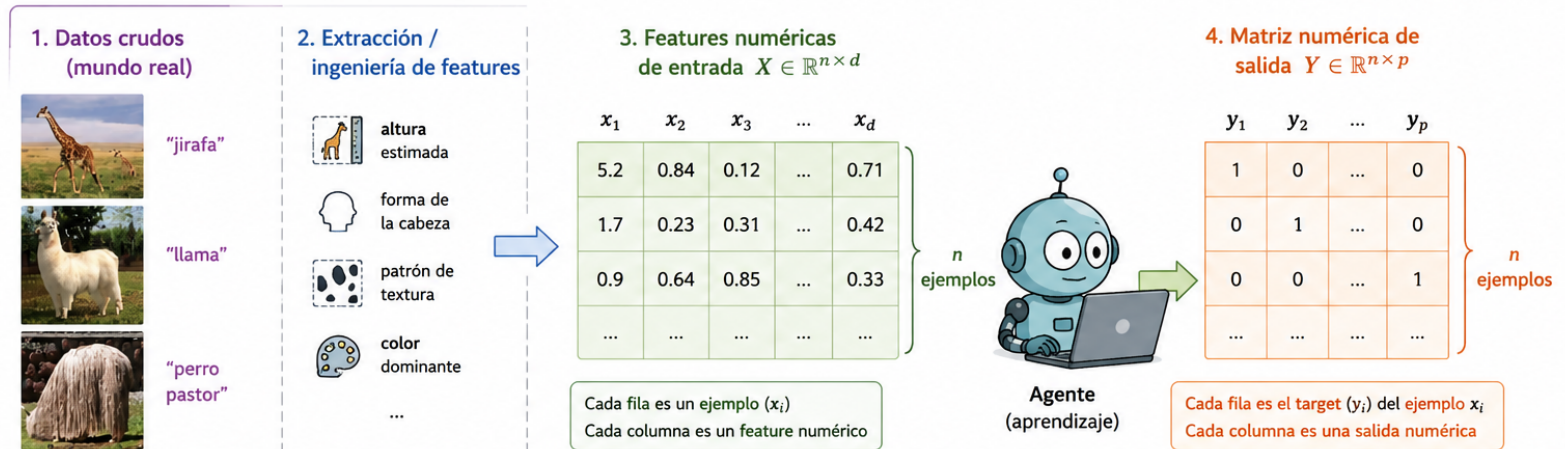
$\approx$ \$487 millones
?



En regresión, la salida $f(x)$ es un número real.
Buscamos predecir **valores continuos**.

Características

Convertimos datos del mundo real en números (features) para que el agente pueda aprender



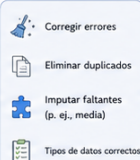
Pre-procesamiento

1 Limpieza e imputación

Corregimos errores, eliminamos duplicados y completamos valores faltantes.

Datos crudos (con problemas)

precio (MUSD)	área (m²)	hab.	ubicación	antigüedad (años)
0.50	120	3	Norte	10
?	200	4	Sur	-
0.80	-	3	Norte	5
0.65	150	?	Este	15
1.20	250	4	Sur	-



Después de limpieza e imputación

precio (MUSD)	área (m²)	hab.	ubicación	antigüedad (años)
0.50	120	3	Norte	10
0.80	200	4	Sur	12.5
0.80	150	3	Norte	5
0.65	150	3	Este	15
1.20	250	4	Sur	12.5

Objetivo: datos consistentes, completos y listos para ser transformados en features.

2 Codificación de variables categóricas

Convertimos variables categóricas en representaciones numéricas.

Dato categórico

ubicación
Norte
Sur
Este
Sur
Norte



Representación numérica (one-hot)

Norte	Sur	Este
1	0	0
0	1	0
0	0	1
0	1	0
1	0	0

También existen codificaciones ordinales, por conteo, target encoding, etc.

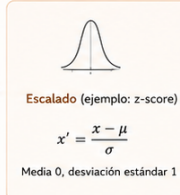
Objetivo: que el algoritmo pueda trabajar con categorías como números.

3 Escalado / Normalización

Ponemos las features en escalas comparables para mejorar el aprendizaje.

Datos después de limpieza y codificación

precio (MUSD)	área (m²)	hab.	antigüedad (años)	Norte	Sur	Este
0.50	120	3	10	1	0	0
0.80	200	4	12.5	0	1	0
0.80	150	3	5	1	0	0
0.65	150	3	15	0	1	0
1.20	250	4	12.5	0	1	0



Datos después de escalado

precio (z)	área (z)	hab. (z)	antigüedad (z)	Norte	Sur	Este
-1.12	-1.07	0.00	-0.59	1	0	0
-0.27	0.97	1.41	0.00	0	1	0
-0.27	-0.27	0.00	-1.18	1	0	0
-0.71	-0.27	0.00	1.19	0	1	0
2.10	1.61	1.41	0.59	0	1	0

Otras opciones: min-max (escala [0,1]) o robust scaler (usa mediana e IQR).

Objetivo: que todas las features contribuyan en la misma magnitud al modelo.

Preguntas básicas

- ¿Qué espacio de hipótesis H elegir?
- ¿Cómo medir el grado de ajuste?
- ¿Cómo equilibrar ajuste vs. complejidad? — la navaja de Occam.
- ¿Cómo encontramos una buena h ?
- ¿Cómo sabemos si una buena h va a predecir bien sobre datos nuevos?

Sub/Sobre ajuste

Overfitting (sobreajuste)

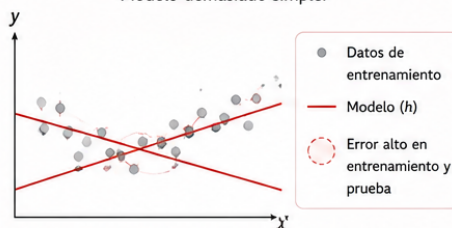
El modelo aprende demasiado los datos de entrenamiento y generaliza mal a datos nuevos.



No es solo explicar los datos de entrenamiento, sino generalizar.

1. Underfitting (subajuste)

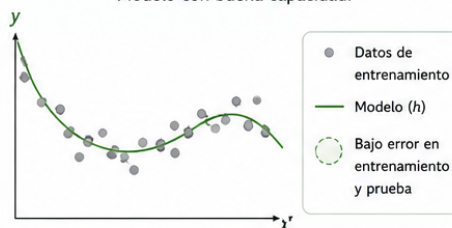
Modelo demasiado simple.



No captura el patrón de los datos.

2. Buen ajuste

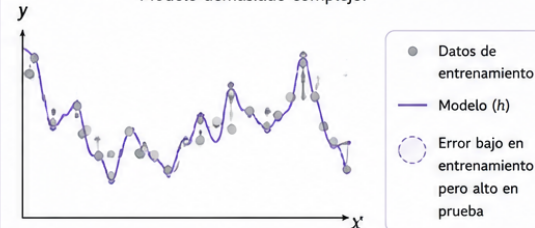
Modelo con buena capacidad.



Captura el patrón y generaliza bien.

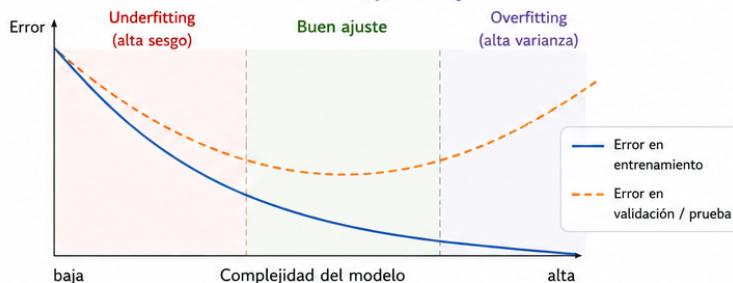
3. Overfitting (sobreajuste)

Modelo demasiado complejo.



Se ajusta al ruido y falla en datos nuevos.

Curvas de aprendizaje



¿Cómo detectarlo?



El error de entrenamiento sigue bajando, pero el error de validación empieza a subir.



Gran brecha entre ambos errores.



El modelo funciona muy bien con los datos que ya vio, pero mal con datos nuevos.

¿Qué hacer?

- ✓ Simplificar el modelo
- ✓ Regularización (L1, L2)
- ✓ Más datos de entrenamiento
- ✓ Early stopping
- ✓ Validación cruzada



Idea clave



Datos de entrenamiento



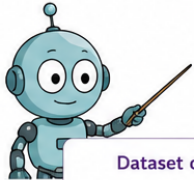
Aprende una función h



Meta: buen desempeño en datos nuevos

Queremos un modelo que encuentre el patrón verdadero, no que memorice.

Validación cruzada



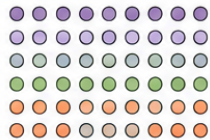
Particiones de datos: Entrenamiento, Validación y Prueba

Dividimos los datos para entrenar el modelo, ajustar sus decisiones y estimar su desempeño de forma justa.

Dataset completo

$$D = \{(x_i, y_i)\}_{i=1}^N$$

N ejemplos



Características X
Etiquetas/objetivos y

1. Partición típica (hold-out)

Entrenamiento	Validación	Prueba (test)
60% – 80%	10% – 20%	10% – 20%

Se usa para aprender los parámetros del modelo.

El modelo NO debe evaluarse aquí (riesgo de sobreajuste).

Se usa para ajustar decisiones del modelo y seleccionar hiperparámetros.

Se puede evaluar muchas veces.

Se usa una sola vez al final para estimar el desempeño de generalización.

Debe permanecer intocado hasta el final.



Regla de oro: el modelo solo puede "ver" el conjunto de prueba al final.

2. ¿Cómo se usan?



Entrenar modelo h_θ con datos de entrenamiento.



Ajustar hiperparámetros usando el conjunto de validación (elegir el mejor modelo).



Evaluar el modelo final en el conjunto de prueba y reportar el desempeño.

3. Validación cruzada (k -fold)

Útil cuando el dataset es pequeño. Permite usar todos los datos para entrenar y validar.

Ejemplo: $k = 5$

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Iteración 1	Validación	Entrenamiento	Entrenamiento	Entrenamiento	Entrenamiento
Iteración 2	Entrenamiento	Validación	Entrenamiento	Entrenamiento	Entrenamiento
Iteración 3	Entrenamiento	Entrenamiento	Validación	Entrenamiento	Entrenamiento
Iteración 4	Entrenamiento	Entrenamiento	Entrenamiento	Validación	Entrenamiento
Iteración 5	Entrenamiento	Entrenamiento	Entrenamiento	Entrenamiento	Validación

■ Entrenamiento ■ Validación

Resultado

Promediamos el desempeño en las 5 iteraciones para obtener una estimación más estable.

$$Score_{CV} = \frac{1}{k} \sum_{i=1}^k Score_i$$

donde $Score_i$ es el desempeño en la iteración i .



La validación cruzada se usa para selección de modelos/hiperparámetros.

Al final, entrenamos el modelo definitivo con entrenamiento + validación y evaluamos en prueba.

4. Resumen

Conjunto	Uso principal	Se toca durante el desarrollo?	Objetivo
Entrenamiento	Aprender parámetros del modelo	Sí	Minimizar error de entrenamiento
Validación	Ajustar modelo / seleccionar hiperparámetros	Sí (muchas veces)	Elegir el mejor modelo
Prueba (test)	Evaluación final	No (una sola vez al final)	Estimar desempeño de generalización real

Buenas prácticas

- ✓ Particionar aleatoriamente (idealmente estratificado si hay clases desbalanceadas).
- ✓ Asegurar que no haya fuga de información entre particiones.
- ✓ Mantener el conjunto de prueba intocado hasta el final.
- ✓ Reportar el desempeño en el conjunto de prueba solo una vez.



El objetivo no es memorizar los datos, sino generalizar a datos nuevos y nunca vistos.

Ejemplo: filtro de spam

Entrada: un correo electrónico

Salida: spam / ham

¿Cómo aprende?

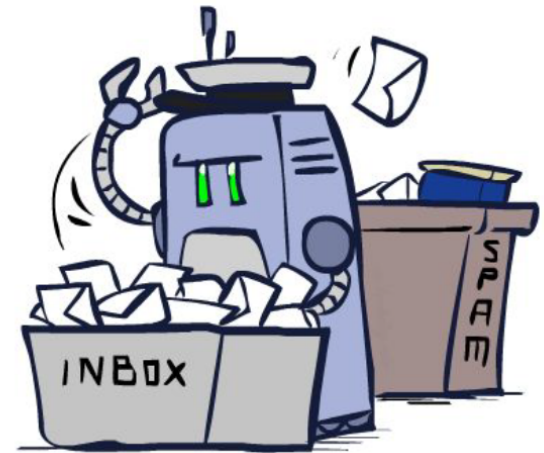
Coleccionamos muchos correos etiquetados manualmente y aprendemos a predecir la etiqueta de **correos nuevos**.

Features

El correo debe convertirse en atributos que el modelo pueda utilizar:

- **Palabras:** FREE !, WINNER, URGENT
- **Patrones:** \$\$\$, MAYÚSCULAS, muchos !
- **Metadatos:** remitente conocido o desconocido
- **Enlaces:** texto del enlace $\neq$ destino real

(x_i, y_i)



Los filtros de spam rechazan alrededor de **200 mil millones de correos spam al día**.

Ejemplo: reconocimiento de dígitos

¿Cómo aprende?

Usamos un conjunto de ejemplos etiquetados:

MNIST: 60 mil imágenes de entrenamiento.

El objetivo es predecir correctamente el dígito de una **imagen nueva**.

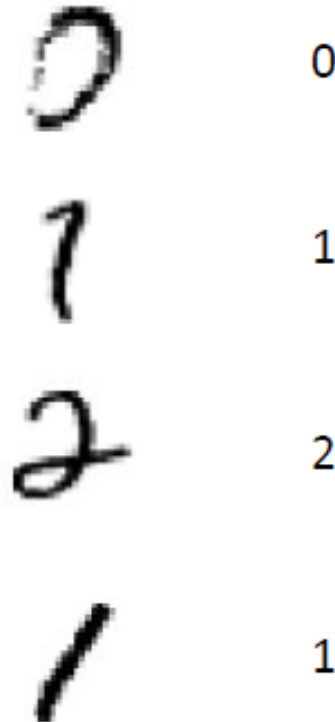
Features

La imagen debe representarse mediante atributos numéricos:

- **Píxeles:** intensidad en cada posición
- **Forma:** proporciones, componentes, número de loops
- **Patrones locales:** bordes, trazos, curvas
- **Representaciones aprendidas:** mapas de activación / filtros

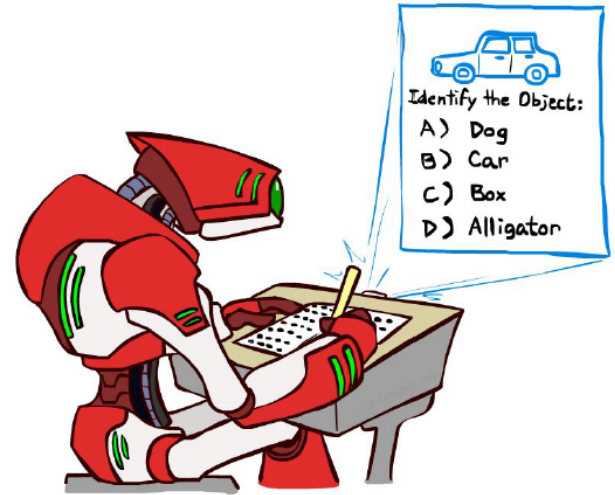
Entrada: imagen / grilla de píxeles

Salida: un dígito entre **0 y 9**



Otras tareas de clasificación

- **Diagnóstico médico:**
síntomas → enfermedad
- **Calificación automática de ensayos:**
documento → categoría / nota
- **Detección de fraude:**
actividad de cuenta → fraude / no fraude
- **Enrutamiento de correo:**
queja de cliente → departamento
- **Inspección de alimentos:**
imagen o sensores → mohoso / OK



En todos los casos:

$$x \longrightarrow h(x) \longrightarrow \hat{y}$$

donde $\hat{y}$ pertenece a un conjunto de **categorías posibles**.

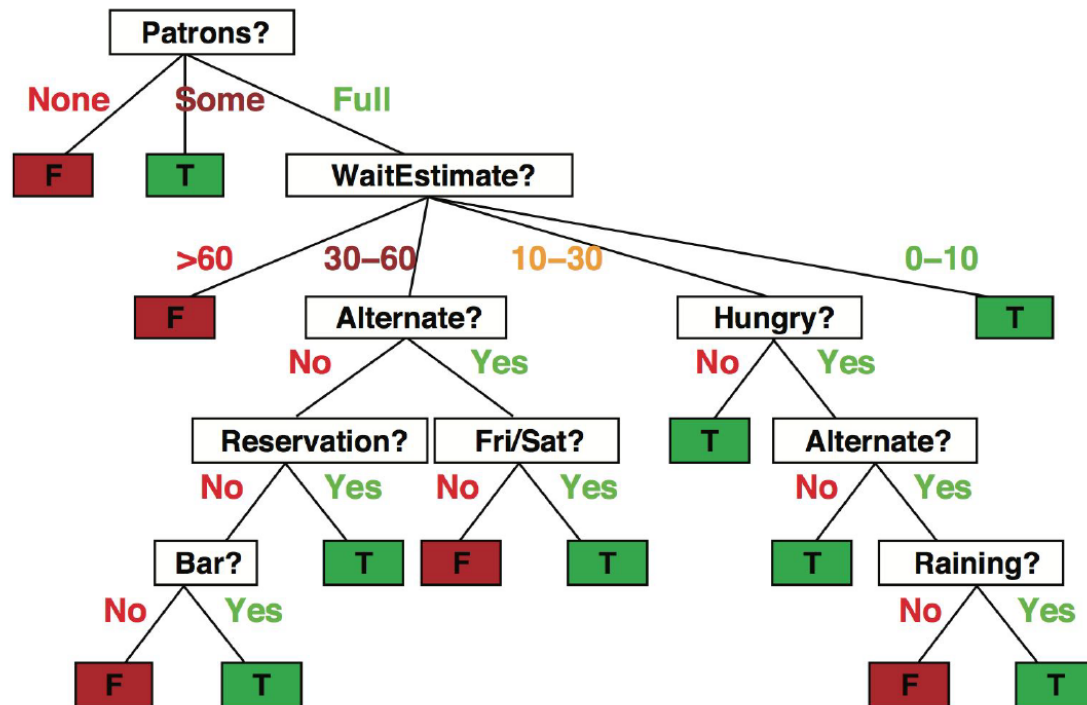
Árboles de Decisión

Modelos de árbol • Construcción del árbol • Medir el desempeño

Árboles de decisión

Representación popular para clasificadores — ¡incluso entre humanos!

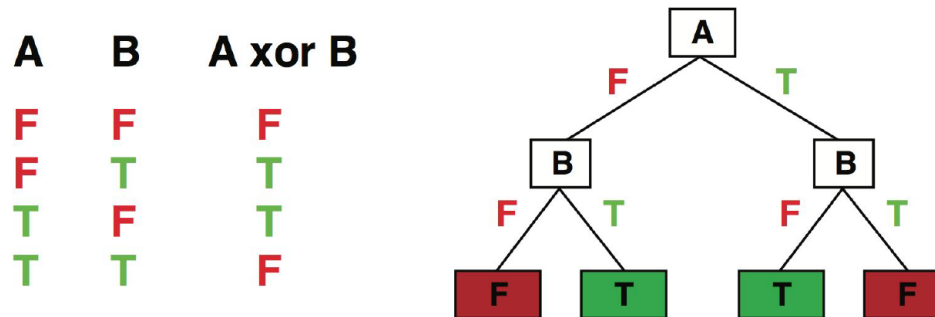
Acabo de llegar a un restaurante: ¿debería esperar por una mesa o irme a otro lado?



Expresividad

Los árboles de decisión discretos pueden expresar cualquier función de la entrada.

Ej.: para funciones booleanas, se construye un camino de la raíz a una hoja por cada fila de la tabla de verdad ($A \text{ xor } B$):



Trivialmente existe un árbol de decisión consistente que ajusta exactamente cualquier conjunto de entrenamiento (a menos que la función real sea no-determinista). Pero un árbol que solo memoriza los ejemplos es, en esencia, una tabla de búsqueda. Para generalizar a ejemplos nuevos necesitamos un árbol **compacto**.

Datos de entrenamiento

Example	Input Attributes										Goal
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	$y_1 = \text{Yes}$
x_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	$y_2 = \text{No}$
x_3	No	Yes	No	No	Some	\$	No	No	Burger	0-10	$y_3 = \text{Yes}$
x_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	$y_4 = \text{Yes}$
x_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	$y_5 = \text{No}$
x_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	$y_6 = \text{Yes}$
x_7	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	$y_7 = \text{No}$
x_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	$y_8 = \text{Yes}$
x_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	$y_9 = \text{No}$
x_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	$y_{10} = \text{No}$
x_{11}	No	No	No	No	None	\$	No	No	Thai	0-10	$y_{11} = \text{No}$
x_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	$y_{12} = \text{Yes}$

Aprendizaje de árboles de decisión

Decision-Tree-Learning

CASOS BASE

```
if examples is empty
    return Plurality-Value(parent_examples)

else if all examples have the same classification
    return classification

else if attributes is empty
    return Plurality-Value(examples)
```

PASO RECURSIVO

```
else
    A ← argmax Importance(a, examples)

    tree ← new decision tree with root test A

    for each value v of A
        exs ← examples where A = v

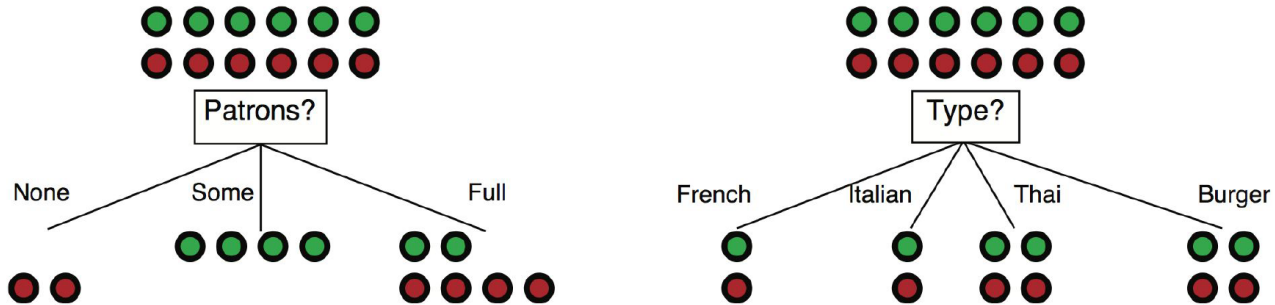
        subtree ← Decision-Tree-Learning(
            exs,
            attributes - {A},
            examples
        )

        add branch (A = v) → subtree

    return tree
```

Eligiendo un atributo: ganancia de información

Idea: medir la contribución de un atributo a aumentar la "pureza" de las etiquetas en cada subconjunto de ejemplos.



Patrons es mejor elección: da *información* sobre la clasificación (reduce la **entropía** de la distribución de etiquetas). **Type** no reduce nada la incertidumbre.

Información y entropía

Idea

La información mide cuánto reduce nuestra **incertidumbre**.

- Si casi sabemos la respuesta $\rightarrow$ poca información.
- Si estamos muy inseguros $\rightarrow$ mucha información.

Entropía multiclase

Para K clases con probabilidades

$\langle p_1, \dots, p_K \rangle$:

$$H(Y) = - \sum_{k=1}^K p_k \log_2 p_k$$

Caso binario

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

Intuición binaria

$$(0.5, 0.5) \rightarrow H = 1$$

Máxima incertidumbre

$$(0.9, 0.1) \rightarrow H \approx 0.47$$

$$(1, 0) \rightarrow H = 0$$

Multiclase

$$\left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right) \rightarrow H = \log_2(3) \approx 1.58$$

Más incertidumbre

$\Rightarrow$ mayor entropía

Ganancia de información

Antes de dividir

Tenemos un conjunto E con clases mezcladas:

$$H(E) = - \sum_c p(c) \log_2 p(c)$$

Ejemplo: 6 positivos y 6 negativos

$$H(E) = H(0.5, 0.5) = 1 \text{ bit}$$

Después de dividir por A

A separa los ejemplos en $E_1, \dots, E_k$.

La entropía después del split es el **promedio ponderado** de las ramas:

$$H(E | A) = \sum_k \frac{|E_k|}{|E|} H(E_k)$$

Ganancia de información

$$\text{Gain}(A) = H(E) - H(E | A)$$

¿Cuánta incertidumbre
eliminó el atributo?

Gain alto

→ ramas más puras

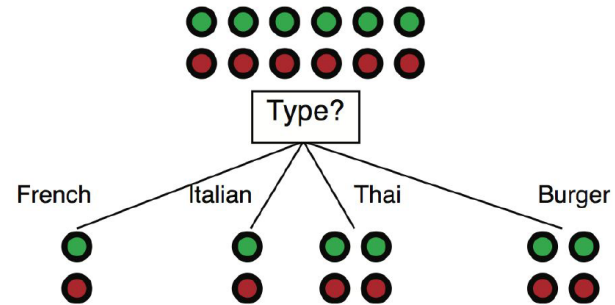
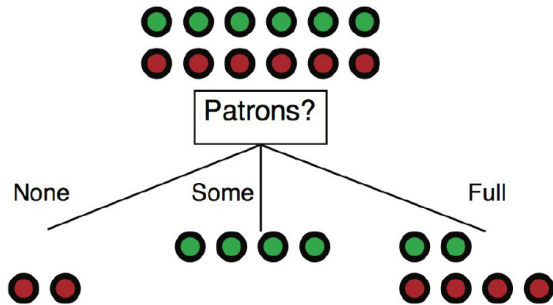
Gain bajo

→ poca información

Gain = 0

→ no mejoró la separación

Ejemplo



Para **Patrons**:

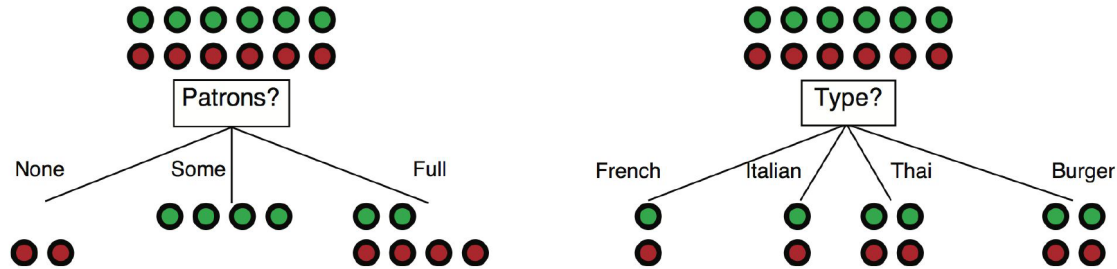
$$1 - \left[\frac{2}{12} B(0) + \frac{4}{12} B(1) + \frac{6}{12} B(2/6) \right] = 0.541 \text{ bits}$$

Para **Type**:

$$1 - \left[\frac{2}{12} B\left(\frac{1}{2}\right) + \frac{2}{12} B\left(\frac{1}{2}\right) + \frac{4}{12} B\left(\frac{2}{6}\right) + \frac{4}{12} B\left(\frac{1}{2}\right) \right] = 0 \text{ bits}$$

Patrons gana casi toda la información posible en un solo paso; Type no gana nada.

Resultado sobre los datos del restaurante



Árbol aprendido a partir de los 12 ejemplos: **más simple** que el árbol "verdadero" que vimos al inicio.

Entrenamiento y prueba



Conceptos básicos

Datos: instancias etiquetadas (ej. episodios de restaurante):

- **Conjunto de entrenamiento**
- **Conjunto de validación**
- **Conjunto de prueba**

Ciclo de experimentación:

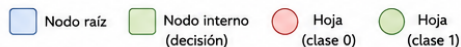
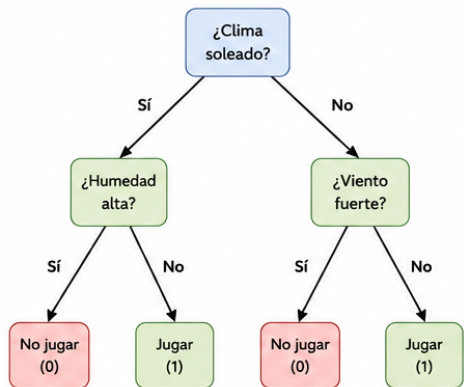
- Generar una hipótesis h a partir del conjunto de entrenamiento.
- (Posiblemente elegir la mejor h probándola en el conjunto de validación.)
- Finalmente, calcular la precisión de h sobre el conjunto de prueba.
- Muy importante: **nunca espiar el conjunto de prueba!**

Evaluación:

- **Precisión:** fracción de instancias predichas correctamente.
- **Curva de aprendizaje:** precisión sobre el conjunto de prueba en función del tamaño del conjunto de entrenamiento.

Hiperparámetros árboles de decisión

ÁRBOL DE DECISIÓN



PARÁMETROS PRINCIPALES



max_depth

Profundidad máxima permitida del árbol.



min_samples_split

Mínimo de muestras en un nodo para poder dividirlo.



min_samples_leaf

Mínimo de muestras que debe tener una hoja (nodo final).



max_features

Número de variables a considerar para buscar el mejor split.



ccp_alpha

Parámetro de poda por complejidad (post-poda).



class_weight

Pesos para las clases (útil en datos desbalanceados).

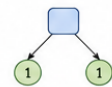
Otros: **criterion** (gini, entropy), **splitter** (best, random), **random_state**

CRITERIOS DE PARADA MÁS USADOS

1

Nodo puro

Todos los ejemplos del nodo pertenecen a la misma clase (impureza = 0). No se puede ganar más información.

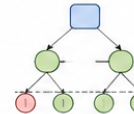


Se detiene.

2

Profundidad máxima alcanzada

Se alcanza el valor de max_depth definido por el usuario.

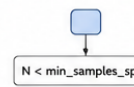


Se detiene.

3

Mínimo de muestras para dividir

El número de muestras en el nodo es menor que min_samples_split. No se divide.

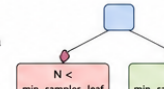


Se detiene.

4

Mínimo de muestras en hoja

Cualquiera de los nodos hijos resultaría con menos de min_samples_leaf muestras.



Se detiene.

5

Sin mejora significativa

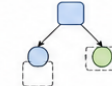
La mejor ganancia de información (o reducción de impureza) es menor que un umbral mínimo (min_impurity_decrease).

Ganancia < umbral
se detiene.

6

Poda por complejidad (opcional)

Usando ccp_alpha, si la reducción en el error no justifica el aumento de complejidad, se poda y se detiene.



Se poda y se detiene.

EL AGENTE APRENDE DEL PASADO Y PREDICE SOBRE NUEVOS EJEMPLOS



Datos del mundo real

- Imágenes
- Tablas
- Sensores
- ...



Extracción / ingeniería de features

- Medir
- Transformar
- Seleccionar



Matriz numérica de entrada $X \in \mathbb{R}^{n \times d}$

x_1	x_2	...	x_d
5.2	0.84	...	0.71
1.7	0.23	...	0.42
...	...	...	...

n ejemplos



Agente (árbol de decisión)



Aprende la mejor estructura de árbol y reglas de decisión.



Matriz numérica de salida $Y \in \mathbb{R}^{n \times p}$

y_1	y_2	...	y_p
1	0	...	0
0	1	...	0
...	...	...	...

n ejemplos (targets)



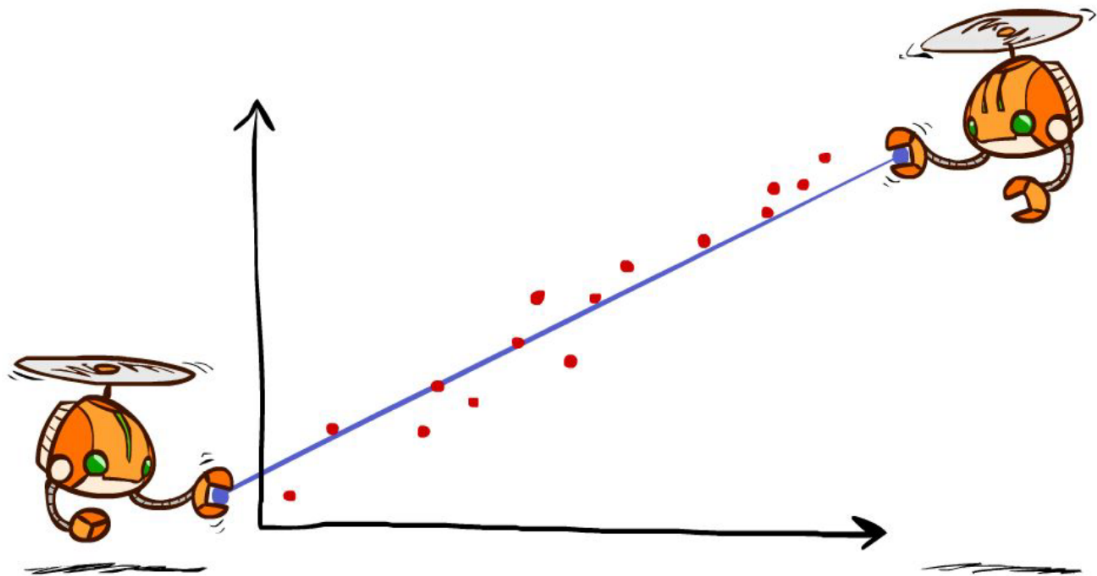
Predice sobre nuevos ejemplos

Entrada: x_{nuevo}
(features numéricos)

Salida: $\hat{y}_{\text{nuevo}}$
(predicción)

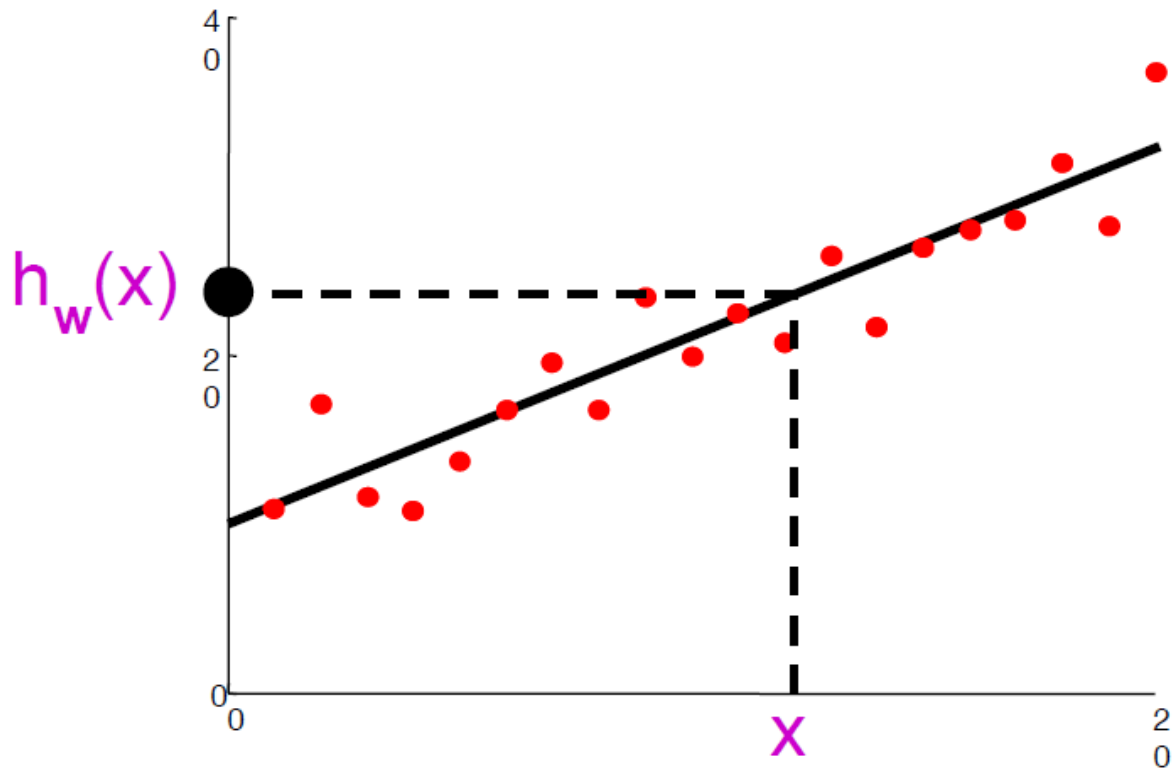


Regresión lineal = ajustar una línea recta / hiperplano



Predicción: $h_w(x) = w_0 + w_1x$

Error de predicción



Prediction: $h_w(x) = w_0 + w_1x$

Regresión lineal: función de pérdida

Para una recta:

$$h_w(x) = w_0 + w_1x$$

medimos el error con la pérdida L_2 :

$$\begin{aligned} L(w_0, w_1) &= \sum_j (y_j - h_w(x_j))^2 \\ &= \sum_j (y_j - (w_0 + w_1x_j))^2 \end{aligned}$$

Objetivo:

$$w^* = \arg \min_w L(w)$$

En el mínimo:

$$\frac{\partial L}{\partial w_0} = 0 \quad \frac{\partial L}{\partial w_1} = 0$$

Esto permite encontrar una **solución exacta**.

Datos $\rightarrow$ ajustar $w_0, w_1 \rightarrow$ menor error

Regresión lineal: solución analítica

Una variable

Para N ejemplos:

$$w_1 = \frac{N \sum_j x_j y_j - (\sum_j x_j)(\sum_j y_j)}{N \sum_j x_j^2 - (\sum_j x_j)^2}$$

$$w_0 = \frac{1}{N} \left[\sum_j y_j - w_1 \sum_j x_j \right]$$

Caso general

Si cada x_j tiene d features:

$$X = [x_1^T, x_2^T, \dots, x_N^T]^T \in \mathbb{R}^{N \times d}$$

Cada **fila** es un ejemplo y cada **columna** es un feature.

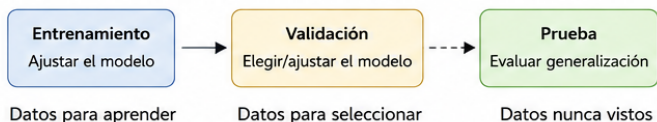
La solución es:

$$w^* = (X^T X)^{-1} X^T y$$

Ecuación normal

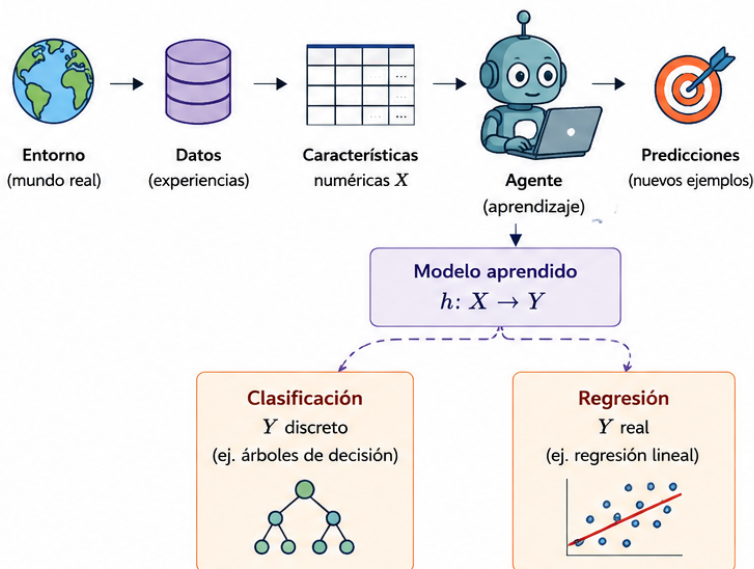
Resumen

- Aprender es esencial en entornos desconocidos, y útil en muchos otros. La naturaleza del proceso depende del diseño del agente, qué pieza se quiere mejorar, y qué datos/conocimiento previo hay disponibles.
- **Aprendizaje supervisado:** aprender una función a partir de ejemplos etiquetados. Hipótesis concisas generalizan mejor — hay que equilibrar concisión y precisión (navaja de Occam).
- **Clasificación:** función de valor discreto — ej. árboles de decisión.
- **Árboles de decisión:** se construyen eligiendo en cada paso el atributo que *maximiza la ganancia de información* (mínima entropía residual). Pueden expresar cualquier función, pero hay que preferir árboles **compactos** para generalizar.
- **Entrenamiento/validación/prueba:** nunca evaluar generalización con datos ya vistos en entrenamiento.



- **Regresión:** función de valor real — ej. regresión lineal, ajustada minimizando el error cuadrático (L_2 loss).

El agente aprende y predice



Principios clave

- ✓ Hipótesis concisas generalizan mejor (navaja de Occam).
- ✓ Elegir el atributo que maximiza la ganancia de información (mínima entropía residual) en árboles de decisión.
- ✓ Preferir modelos compactos para generalizar.
- ✓ Nunca evaluar con datos ya vistos en entrenamiento.
- ✓ En regresión, minimizar el error cuadrático (L_2 loss).

Fin de la Clase 6